

Intransitive Reinforcement Learning Bot

Architecture specification and research roadmap

Version 1.0 | 7 September 2026

Executive recommendation

Build a **solver-augmented AlphaZero system** with a compact, relation-aware convolution–Transformer policy–value network. Train through self-play, use neural-guided search during both training and play, and support it with exact tactical checks and small endgame tablebases. AlphaZero supplies the established search-and-self-play foundation. [1]

System flow: Exact game engine → local/global policy–value network → neural-guided search with tactical proofs → self-play and targeted retraining.

Default model	Search and correctness	Highest-priority research
8 hybrid stages; width 192; about 9–10M parameters	PUCT baseline; Gumbel comparison; exact win/loss checks	Cyclic capture symmetry; solver-supervised forced wins

Status. This is a concrete design proposal, not a benchmarked Intransitive engine. “Optimal” means strongest at the chosen training budget, hardware, and move-time limit. A larger network can weaken the full agent by leaving less time for search.

Contents

1. Game model and rule contract	2
2. Policy–value network	3–4
3. Search, tactical proofs, and endgames	5–6
4. Self-play, losses, and training systems	7–8
5. Research proposals A–D	9–11
6. Evaluation and comparison plan	12
7. Implementation roadmap and model scaling	13
References	14

1. Game model and rule contract

The site identifies the game as **Intransitive**. The supplied instructions show king-like movement, alternating turns with blue first, cyclic captures, and victory by entering the opponent's zone. This specification targets the user-described **9×9 board**. [2]

How to play



User-provided instruction panels. These illustrate the rules, not the full 9×9 playing board.

1.1 Assumptions that the engine must resolve

Assume deterministic, fully observable play with both players' piece types visible. Before training, settle the initial configuration; same-type encounters; whether unfavorable attacks are illegal; elimination and no-legal-move outcomes; repetition and no-progress rules; and exactly when goal entry wins. The browser-readable page did not expose all of these edge cases in the original research. An almost-correct simulator trains an agent for the wrong game.

1.2 Strategic structure

Relational piece value. A piece matters because of the types it threatens, the types that threaten it, and its role in opening or blocking a route. Avoid fixed chess-like material values. **Adversarial reachability.** Distance is not a forced breakthrough; sacrifices and quiet blocks can be decisive. **Tempo and cycles.** Test races, waiting moves, blockades, and repeated positions under the actual draw rules.

1.3 State and action encoding

$$s = (B, p, G_{blue}, G_{red}, \text{rule counters}, \text{relevant history})$$

B is the board, p is the side to move, and the goal sets and history carry all rule-relevant context. This is adversarial planning, not simultaneous rock-paper-scissors.

$$a = 8i + d, i \in \{0, \dots, 80\}, d \in \{0, \dots, 7\}$$

Use 81 source squares × 8 neighboring directions = **648 action slots**. Add pass or draw-claim actions only if supported by the rules. The engine, never the network, determines legal-action masks and terminal outcomes.

2. Policy–value network

Use a convolution–Transformer hybrid: convolutions encode local interactions while full-board attention connects distant pieces and goals. ResTNet reports benefits from interleaving residual and Transformer blocks in Go and Hex, including 9×9 Go. Chessformer/Leela research supports board-appropriate positional representations. These precedents motivate testing; they do not establish performance on Intransitive. [3] [4]

2.1 Default trunk

Component	Specification
Input	Symbolic 9×9 feature planes plus rule/history context
Stem	3×3 convolution into 192 channels
Trunk	8 hybrid stages
Local stage component	Residual block with two 3×3 convolutions
Global stage component	Full-board attention over 81 square tokens
Attention dimensions	6 heads × 32 dimensions
Feed-forward layer	192 → 768 → 192
Normalization / activation	Pre-normalized attention; GELU
Main outputs	Joint legal-move policy; win/draw/loss distribution
Experimental auxiliaries	Action values; bounded-horizon forced-win predictions

The proposed default has approximately **9–10 million parameters**. This is a design estimate, not a published or measured game-specific model.

2.2 Symbolic features

Represent pieces relative to the player to move: “my rock,” “their rock,” and so on. Supply explicit goal masks; do not require the network to infer goals from absolute coordinates.

Feature family	Contents
Occupancy	Six planes: three piece types for each side
Objectives and geometry	Both target zones; valid squares; normalized coordinates
Recent movement	Last move’s source and destination
Tactical relations	Rule-computed, type-specific immediate attack maps
Goal geometry	Empty-board distance to each goal
Rule context / optional history	Applicable repetition and no-progress information; a small number of recent boards

$$d(x, G) = \min_{g \in G} \max(|x_1 - g_1|, |x_2 - g_2|)$$

Chebyshev distance is only a geometric feature; it ignores interference and is not a forced-win estimate. Recent-board planes are only compressed neural input. The engine and search must retain the full relevant repetition state; a few recent boards are not automatically Markov-sufficient.

2. Policy-value network, continued

Relation-aware attention and output heads

2.3 Predator-prey attention bias

$$A_{ij}^{(h)} = \frac{(q_i^{(h)})^\top k_j^{(h)}}{\sqrt{32}} + b_h(\Delta x_{ij}, \Delta y_{ij}) + c_h(\rho_{ij})$$

For attention head h , use a learned relative-position bias b and a learned relationship bias c . The relation p describes empty squares, allies, opposing same-type pieces, attacker-prey pairs, and prey-attacker pairs. The head dimension is 32.

This relation-aware attention is a **proposed extension**, not an established result for Intransitive. The aim is to spend learning capacity on strategic interactions rather than repeatedly rediscovering the capture cycle. Keep attention global: a distant piece may be the only eventual defender of a goal route.

2.4 Joint source-destination policy

Let h_i be the final embedding of square i . For each geometrically possible move $i \rightarrow j$, score the source, destination, their interaction, direction embedding, and piece relationship:

$$\ell_{i \rightarrow j} = \text{MLP}[h_i, h_j, h_i \odot h_j, e_{j-i}, \rho_{ij}]$$

Use a shared MLP with a 128-unit hidden layer as the starting point. Apply a single softmax across the engine-provided legal actions:

$$p_\theta(a \mid s) = \text{softmax}_{a \in A(s)} \ell_a$$

This is one **joint move distribution**, not independent predictions of a source square and direction. Source-destination policy heads have precedent in chess Transformer research; this neighboring-square implementation is adapted to the game's action space. [4]

2.5 Win/draw/loss value head

Pool square embeddings with global mean, global maximum, and goal-region summaries. A small MLP outputs win, draw, and loss probabilities, always from the current player's perspective:

$$p_W + p_D + p_L = 1, v_\theta(s) = p_W - p_L$$

A neutral scalar evaluation can conceal very different cases: a likely draw versus uncertainty between winning and losing. The three-way distribution exposes that distinction, while search uses the scalar difference above.

2.6 Selective auxiliary heads

Action values. Predict $\hat{Q}(s, a)$ for legal moves, using exact results or well-supported search estimates as targets. Initially use this head for supervision and move ordering, not pruning. **Bounded-horizon forced wins.** Predict whether either player can force a win within a specified number of plies; see Proposal B.

Admission rule: Add auxiliary losses one at a time. Keep a head only when a controlled comparison shows stronger play or better learning efficiency.

3. Search design

3.1 Exact transitions before learned dynamics

Use AlphaZero-style planning with the exact simulator. MuZero learns a planning model without being given the environment dynamics; here the rules can provide exact transitions. Learning those transitions would introduce an unnecessary first research problem. [1] [5]

3.2 Establish a PUCT baseline

$$a^* = \underset{a}{\operatorname{argmax}} \left[Q(s, a) + c_{puct} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)} \right]$$

Q is the current action-value estimate, P the network prior, and N the visit count. The exploration constant controls how strongly the search follows prior-weighted, underexplored actions. This conventional baseline provides a reference before introducing alternative search policies.

3.3 Compare Gumbel AlphaZero

Test Gumbel search particularly for compute-limited self-play. It combines root sampling without replacement with sequential halving to allocate root search. Its policy-improvement guarantee is conditional on correctly evaluated action values; it is not a guarantee that an imperfect neural bot strengthens after every update. [6]

Usage	Initial search-budget sweep
Cheap self-play moves	32–64 simulations
High-quality training targets	256–512 simulations
Evaluation	128, 512, 2,048, and 8,192 simulations
Gumbel root candidates	16–32 initially, bounded by the legal-action count

These are experimental starting settings, not known optima. For Gumbel training, use its **improved-policy target**, rather than assuming normalized visit counts are correct. Do not automatically combine Gumbel root exploration with AlphaZero’s Dirichlet-noise recipe.

3.4 Exact root-level tactical checks

First: check every legal move for an immediate win. Select a winning move when one exists. **Second: check the remaining moves for an immediate winning opponent reply.** A legal winning reply is a proof that the candidate move loses, provided the terminal rules are correct.

Exclude proven-losing moves when alternatives remain. Do not prune a move merely because a threat head or shallow heuristic dislikes it. These checks address a likely failure mode of goal-entry play: a globally promising move can still permit an immediate breakthrough.

Compute comparison: Evaluate complete agents under the same wall-clock move limit. Equal simulation counts are diagnostic, not sufficient when networks have different evaluation costs.

3. Tactical proofs and endgames

Keep exact results separate from statistical estimates

3.5 Explicit proof status

Maintain a proof-status field independently of neural values and visit statistics:

Status	Interpretation
UNSOLVED	No exact outcome has been established
PROVEN_WIN	A sound argument establishes a win
PROVEN_DRAW	A sound argument establishes a draw
PROVEN_LOSS	All legal alternatives have been ruled out as non-losing

A neural value of **0.999 is not a proof**. At nonterminal positions, a winning continuation can establish a win for the mover; proving a loss requires checking every legal alternative. Searching only a Gumbel-selected subset of root moves therefore cannot, by itself, prove that the position is lost.

MCTS-Solver provides a foundation for separating solved outcomes from Monte Carlo estimates. Proof-number methods are a further direction for proof-oriented exploration. Add a bounded alpha-beta or proof-number worker around decisive goal threats; selective move ordering is acceptable, but a reported proof must remain sound. [7]

3.6 Small endgame tablebases

Start with two-piece endings, then selected three-piece endings. Use exact outcomes in search, as supervised training examples, and as a test suite for basic races and blockades. Enlarge the tables only after measuring their value.

History caveat: A board-only tablebase is not automatically an exact oracle for every repetition history. Its state representation and applicability checks must match the actual draw rules.

3.7 Conservative transposition handling

Initially reuse neural evaluations while keeping search visit statistics local to each parent edge. Full graph-search sharing is a separate implementation project, not a free substitution for tree search.

Cache keys must include the relevant rule context and network version. Search must stop cycles from producing endless simulations. When values are stored from the side-to-move perspective, negate them consistently across alternating turns.

3.8 Correctness gates

Treat legal moves, terminal outcomes, repetition handling, proof propagation, and value-sign conventions as tested engine invariants. Exact tactical checks and endgame oracles should expose neural mistakes, not inherit errors from an unverified rules implementation.

4. Self-play and training design

4.1 Optimize winning, not proxy objectives

$$z \in \{-1, 0, +1\}, \gamma = 1$$

Use terminal loss, draw, and win outcomes with an undiscounted objective. Do not reward captures, material advantage, or advancing toward the goal in the final objective: these proxies can conflict with winning. Such information can be an input feature or auxiliary prediction target without changing the reward.

An administrative training timeout must not silently become a draw unless the rules make it one. Distinguish terminal games from truncated trajectories.

4.2 Starting-position and opponent mixtures

Starting-position source	Initial fraction
Actual game initialization	70%
Previously encountered difficult positions	20%
Verified legal endgame exercises	10%

Separately, start opponent selection at roughly **80% current-agent self-play** and **20% games against a rotating pool of earlier checkpoints and specialist opponents**. Starting-state selection and opponent selection are different sampling decisions.

Keep most training anchored to actual game initialization. Improvement on a convenient curriculum need not translate into stronger normal-start play.

4.3 Randomize search expenditure

An initial recipe is **80% cheap moves at 64 simulations** and **20% expensive moves at 512 simulations**, averaging about **154 simulations per move**. Retain outcome supervision from the games, but preferentially use expensive searches for policy targets or down-weight cheap targets.

This adapts KataGo's playout-cap randomization. KataGo also supplies precedent for auxiliary supervision and other improvements to self-play training. The exact mixture above is a proposed setting for this project. [\[8\]](#)

4.4 Keep training labels interpretable

Store which network generated a position, how much search was used, whether the trajectory ended or was truncated, and whether a target came from a rollout, stronger search, or exact proof. Keep these sources distinguishable during replay and reanalysis.

Curriculum guardrail: Retain separate evaluation on the actual initial position. Do not let a targeted training distribution redefine the game the bot is meant to master.

4. Losses, optimization, and systems

Initial hyperparameters are tuning proposals, not validated optima

4.5 Training objective

$$L = \lambda_{\pi} CE(\pi_{search}, p_{\theta}) + \lambda_v CE(y_{WDL}, v_{\theta, WDL}) + \lambda_Q L_Q + \lambda_F L_{forced-win}$$

$$\lambda_{\pi} = 1, \lambda_v = 1, \lambda_Q = 0.25, \lambda_F = 0.1$$

The main losses are cross-entropy against the search-improved policy and the terminal win/draw/loss target. Add action-value and forced-win losses only once their labels and masks are reliable. Mask action-value targets on insufficiently searched actions. A solver timeout is **unknown**, not a negative forced-win label.

4.6 Optimization defaults

Setting	Initial value
Optimizer	AdamW
Learning rate	2×10^{-4}
Weight decay	1×10^{-4}
Global gradient clipping	1.0
Effective batch size	256–512 positions
Replay capacity	Approximately 1–3 million positions
Replay representation	Compact states and sparse policy targets

Track **training examples consumed per fresh position generated**, not only gradient steps. An oversized learner repeatedly fitting stale replay is not necessarily making useful progress.

Periodically re-search selected old positions with the current network. Keep original outcomes, new search targets, and generating model versions separate so that reanalysis does not obscure target provenance.

4.7 Implementation options

Research prototype. JAX with DeepMind’s Mctx is a practical candidate: it provides batched, JIT-compatible search and Gumbel-policy support. Supply the exact game engine through the transition callback; a learned transition model is not required. [9]

Customized engine. Use a compiled game/search implementation with a batched neural inference service when tighter control is needed. Batch across many concurrent games, while keeping latency low enough for time-limited play.

System-level objective: Optimize strength per unit of training compute and per millisecond of decision time, not isolated network throughput or loss alone.

5. Research proposals

These are testable proposals, not claims of established novelty or guaranteed improvement. Their contribution would be the formulation, implementation, and evidence in this setting, rather than inventing equivariance, proof search, or self-play from scratch.

A. Cyclic-symmetry-aware predator-prey networks

Hypothesis. Enforcing the capture cycle's symmetry reduces the data required to learn strategically equivalent interactions.

Provided piece types differ only through the capture relation, the following global relabeling preserves that directed relation:

$$R \rightarrow P, P \rightarrow S, S \rightarrow R$$

The three cyclic relabelings form C_3 . This is **not** permission to use all six permutations: an arbitrary type swap reverses part of the directed relationship.

Variant	Mechanism
Augmentation	Randomly apply valid cyclic type relabelings during training
Consistency training	Penalize disagreement between equivalent transformed states
Exact weight sharing	Maintain a three-element type axis and tie weights cyclically

For a verified symmetry g , the policy should transform with the actions while the value remains unchanged:

$$p_{\theta}(ga \mid gs) = p_{\theta}(a \mid s), v_{\theta}(gs) = v_{\theta}(s)$$

Group-equivariant networks provide the established general foundation. The game-specific research question is whether enforcing this directed capture symmetry improves strength per unit compute. [10]

Validation and risks

Do not blindly impose every rotation or reflection of the square board. Goal locations, player roles, and other rules must transform correctly. Test the transition identity before admitting a transformation:

$$T(gs, ga) = gT(s, a)$$

Success criterion. Higher strength at matched training compute and better performance on held-out piece arrangements, not merely lower symmetry error. **Priority.** High. Begin with augmentation before engineering exact equivariant layers.

5. Research proposal B

Highest-priority game-specific research direction

B. Solver-supervised adversarial reachability

Hypothesis. Learning whether a win can be forced is more useful than learning only geometric goal distance or typical self-play outcomes.

For a fixed player p , define a bounded-horizon forced-win target:

$$W_h^p(s) = 1 \{ \text{player } p \text{ can force a win within } h \text{ plies} \}$$

With terminal cases handled by the official rules, the recursion for nonterminal states is:

$$W_h^p(s) = \max_{a \in A(s)} W_{h-1}^p(T(s, a)), \text{ if } p \text{ is to move}$$

$$W_h^p(s) = \min_{a \in A(s)} W_{h-1}^p(T(s, a)), \text{ if the opponent is to move}$$

The minimum over opponent replies is essential: following one plausible continuation does not prove a breakthrough. A ply is one player's move.

Training design

Generate exact labels on tractable states at horizons such as **1, 2, 4, and 8 plies**. Train a compact multi-horizon head and encourage consistency with:

$$W_h^p(s) \leq W_{h+1}^p(s)$$

Use predictions to prioritize search, while the exact solver retains responsibility for certificates. MCTS-Solver and proof-number methods supply the search foundation; the proposed contribution connects exact tactical reasoning to a learned multi-horizon representation. [7]

Evaluation and resource allocation

Success criterion. Fewer missed forced wins and forced defenses, followed by higher match strength at equal wall-clock time. **Risk.** Exact labels can be expensive. Reserve deeper labeling for small endings, near-goal situations, and positions where the model is uncertain.

Soundness requirement: An unproven or timed-out state must remain unknown. A neural prediction is useful for ordering and supervision, but it cannot establish a proven win or loss.

Priority. Highest. This proposal targets the actual win condition rather than relying on a general architectural trend.

5. Research proposals C and D

C. Regret-guided curriculum and adversarial opponents

Hypothesis. Revisiting consequential mistakes is more valuable than spending all self-play compute on ordinary openings. Regret-Guided Search Control, an ICLR 2026 work, learns to identify high-regret states and reuses self-play and search positions as starting points, reporting gains in Go, Othello, and Hex. [11]

Prioritize positions where stronger search or an exact solver substantially changes the value or preferred move. One proposed priority signal is:

$$r(s) = | v_{teacher}(s) - v_{student}(s) |$$

This is a candidate signal for this project, not necessarily the paper's exact regret definition. Combine targeted restarts with a small population of opponents aimed at goal rushing, blockade defense, sacrifices, and repeated-position behavior.

Adversarial-policy research on Go shows that very strong agents can retain exploitable weaknesses. Test robustness separately from average strength, including against newly trained exploiters rather than only a fixed pool. [12]

Success criterion. Better results against both the ordinary pool and fresh exploiters. **Risk.** Overfitting to a narrow adversary or unusual curriculum. Preserve substantial normal-start self-play and untouched evaluation data.

Priority. High after the baseline is stable.

D. Adaptive internal reasoning versus tree search

Hypothesis. Difficult states may benefit from extra neural refinement, but only when its value exceeds the external search it displaces. Replace part of the fixed-depth trunk with a shared refinement block:

$$H^{(k+1)} = F_{\theta}(H^{(k)}, E(s))$$

Train with variable iteration counts, such as 2, 4, and 8, and supervise intermediate outputs. This is recurrence within one position's evaluation, not a replacement for the game engine's history.

The 2025 Tiny Recursive Model work reports effective small recursive networks on difficult puzzles. It does not establish that recurrence improves competitive board-game engines; this remains an experiment. [13]

Extend the idea with a controller that chooses another neural refinement, another search expansion, or stopping. Train against stronger reference search while balancing decision error against computation.

Success criterion. Stronger play at equal milliseconds per move. **Risk.** Better evaluation can still reduce complete-agent strength by cutting search throughput. Confidence or low policy entropy alone is not a safe stopping rule. **Priority.** Medium; more speculative than tactical supervision and targeted self-play.

6. Evaluation and comparison plan

6.1 Controlled agent sequence

Agent	Question being tested
Residual CNN + PUCT	Basic AlphaZero reference
Hybrid trunk + PUCT	Local/global architectural benefit
Hybrid trunk + Gumbel	Search-policy improvement
Best variant + tactical checks	Value of exact tactical correctness
Best variant + cyclic symmetry	Value of exploiting type symmetry
Best variant + forced-win head	Value of adversarial reachability
Best variant + targeted curriculum	Learning efficiency and robustness
Adaptive-depth variant	Internal reasoning versus external search

Include pure Transformer and CPU-oriented alpha-beta alternatives when resources permit. For CPU deployment, a small NNUE-style evaluator with alpha-beta is a relevant comparison: Stockfish demonstrates the engineering value of an efficiently updatable neural evaluator, but unmodified Stockfish cannot be assumed to play this game. [14]

A modern model-free reference is also useful. The February 2026 KLENT paper reports resource-efficient learning across five board games. Include it in low-training-budget comparisons without assuming that its results establish the highest possible strength here. [15]

6.2 Evaluation axes

Playing strength. Use paired games with agents assigned both blue and red, a diverse opponent pool, and confidence intervals clustered by opening or starting position. Keep standard-start results separate from midgame-start and puzzle results.

Tactical reliability. Measure missed immediate wins, allowed immediate losses, solved-endgame errors, and forced-win/defense accuracy at multiple depths.

Compute efficiency. Report strength against training compute and actual move time. Equal simulation counts do not make differently sized networks a fair deployment comparison.

Robustness. Test freshly trained exploiters and held-out position families. Retain the full matchup matrix: a single Elo number can hide cyclic strengths and weaknesses.

6.3 Experimental discipline

Repeat key training comparisons with multiple seeds. Split data by trajectories or position families and account for symmetries so equivalent states do not leak across training and testing. Compare whole agents, not network losses in isolation.

7. Implementation roadmap

7.1 Build in this order

Phase 1: correctness. Build the exact engine, exhaustive small-position move-generation tests, transition tests, and a simple minimax opponent. Compare transitions directly with the website wherever possible.

Phase 2: reference agent. Train a smaller residual policy-value network using ordinary self-play and PUCT. Use it to uncover simulator, masking, repetition, and value-sign errors and to establish a trustworthy opponent.

Phase 3: production candidate. Move to the 192-channel hybrid, add relation-aware attention, compare Gumbel and PUCT, and integrate immediate-threat checks and small endgame oracles.

Phase 4: research increments. Add cyclic augmentation, solver-supervised forced-win heads, regret-guided restarts, and adversarial training individually. Test adaptive recurrent depth only after these foundations are stable.

7.2 Scale the full agent, not just the network

Scale	Trunk	Approximate parameters
Fast	6 hybrid stages; width 128	3M
Default	8 hybrid stages; width 192	9–10M
Large	12 hybrid stages; width 256	24M

These are design estimates. Select the winner by complete-agent performance at the intended move-time limit. A better per-position evaluator can lose if it is too slow to search effectively.

7.3 Decisions still needed

The final deployment choice depends on **available training hardware**, **maximum thinking time per move**, and the **fully verified game rules**. These determine network scale, search breadth, batching, tablebase scope, and curriculum cost.

Recommended starting system: A roughly 9M-parameter relation-aware convolution-Transformer policy-value network, trained by search-based self-play, supported by exact tactical checks and endgame oracles, then improved with forced-win supervision and targeted adversarial curricula.

Core research question

Can a network that explicitly models the cyclic capture relation and learns adversarial goal reachability outperform a generic AlphaZero network at equal total compute?

That is a sharper test than asking whether a newer or larger model plays better. The answer should be established through measured agent strength, tactical reliability, robustness, and compute efficiency.

References

Sources cited in the original specification. Numbered citations in the text link to these entries; source labels below open the corresponding publications or official resources. Game-specific settings and research extensions remain proposals.

- [1] **AlphaZero: Shedding new light on chess, shogi and Go.** Google DeepMind. Foundation for policy-value self-play with search. [DeepMind research overview](#)
- [2] **Intransitive.** Game page. Rule edge cases remain subject to engine verification. [Game website](#)
- [3] **ResTNet.** Residual-Transformer architecture research for board-game agents; IJCAI 2025. [arXiv:2410.05347](#)
- [4] **Chessformer / Leela chess Transformer research.** Board-appropriate positional representations and source-destination policy heads. [arXiv:2409.12272](#)
- [5] **MuZero: Mastering Go, chess, shogi and Atari without rules.** Google DeepMind. Planning with a learned model of environment dynamics. [DeepMind research overview](#)
- [6] **Gumbel AlphaZero.** Root sampling, sequential halving, and improved-policy targets. [Research paper \(PDF\)](#)
- [7] **MCTS-Solver.** Exact solved outcomes within Monte Carlo tree search. [Research paper \(PDF\)](#)
- [8] **KataGo self-play training research.** Playout-cap randomization, auxiliary targets, and training efficiency. [arXiv:1902.10565](#)
- [9] **Mctx.** Google DeepMind's batched, JIT-compatible search library. [Official GitHub repository](#)
- [10] **Group Equivariant Convolutional Networks.** Cohen and Welling, 2016. General foundation for symmetry-aware networks. [PMLR, volume 48](#)
- [11] **Regret-Guided Search Control.** ICLR 2026. Targeted reuse of high-regret states for learning. [arXiv:2602.20809](#)
- [12] **Adversarial Policies Beat Superhuman Go AIs.** Adversarial evaluation and exploitable weaknesses in strong game agents. [arXiv:2211.00241](#)
- [13] **Tiny Recursive Model research.** 2025 research on small recursive networks for difficult puzzles. [arXiv:2510.04871](#)
- [14] **Stockfish NNUE documentation.** Efficiently updatable neural evaluation for CPU-oriented search. [Official NNUE documentation](#)
- [15] **KLENT.** February 2026 model-free board-game learning research. [arXiv:2602.10894](#)